

ORDER AND ORDER DETAILS MANAGEMENT SYSTEM

PURPLLE ONLINE BEAUTY SHOPPING

1. INTRODUCTION

The **Purple Online Beauty Shopping Database Management System** is designed to manage customer orders and order details. The **Orders** and **Order_Details** tables store order information such as order date, quantity, price, and total amount.

2. OBJECTIVES

- Design **Orders** and **Order_Details** tables.
- Manage customer product orders.
- Store order date, quantity, and total amount.
- Perform order insertion and modification.
- Generate customer order history reports.
- Generate customer-wise order summaries.

3. DATABASE DESIGN

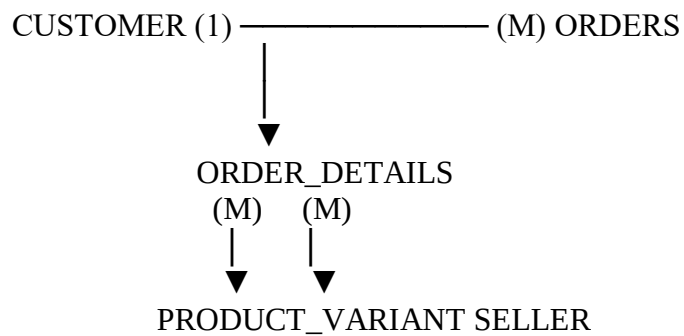
3.1 ORDERS TABLE

Attribute	Data Type	Key	Description
Order_ID	INT	PK	Unique order ID
Customer_ID	INT	FK	References Customer
Coupon_ID	INT	FK	References Coupon, if applied
Order_Date	DATE	-	Date of order
Total_Amount	DECIMAL(10,2)	-	Total order amount
Order_Status	VARCHAR(30)	-	Pending/Shipped/Delivered/Returned

3.2 ORDER DETAILS TABLE

Attribute	Data Type	Key	Description
Order_Detail_ID	INT	PK	Unique order detail ID
Order_ID	INT	FK	References Orders
Variant_ID	INT	FK	References Product_Variant
Seller_ID	INT	FK	References Seller fulfilling this line item
Quantity	INT	-	Ordered quantity
Unit_Price	DECIMAL(10,2)	-	Price per unit at time of order

4. TABLE RELATIONSHIPS



Customer_ID (PK) → Customer_ID (FK)

Order_ID (PK) → Order_ID (FK)

Variant_ID (PK) → Variant_ID (FK)

Seller_ID (PK) → Seller_ID (FK)

5. TABLE CREATION

5.1 CREATE ORDERS TABLE

```
SQL> CREATE TABLE Orders (  
2     Order_ID NUMBER PRIMARY KEY,  
3     Customer_ID NUMBER NOT NULL,  
4     Coupon_ID NUMBER,  
5     Order_Date DATE,  
6     Total_Amount NUMBER(10,2),  
7     Order_Status VARCHAR2(30),  
8     FOREIGN KEY (Customer_ID) REFERENCES Customer5(Customer_ID)  
9 );  
  
Table created.
```

5.2 CREATE ORDER DETAILS

```
SQL> CREATE TABLE Order_Details (  
2     Order_Detail_ID NUMBER PRIMARY KEY,  
3     Order_ID NUMBER NOT NULL,  
4     Product_ID NUMBER NOT NULL,  
5     Seller_ID NUMBER NOT NULL,  
6     Quantity NUMBER NOT NULL CHECK (Quantity > 0),  
7     Unit_Price NUMBER(10,2) NOT NULL CHECK (Unit_Price > 0),  
8     FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),  
9     FOREIGN KEY (Product_ID) REFERENCES PurpleProduct(Product_ID),  
10    FOREIGN KEY (Seller_ID) REFERENCES Seller(Seller_ID)  
11 );  
  
Table created.
```

6. ORDER DATA INSERTION

```
SQL> INSERT INTO Orders VALUES (501, 101, NULL, SYSDATE, 1398.00, 'Pending');
1 row created.

SQL> INSERT INTO Orders VALUES (502, 102, NULL, SYSDATE, 749.00, 'Shipped');
1 row created.

SQL> INSERT INTO Orders VALUES (503, 103, NULL, SYSDATE, 1299.00, 'Delivered');
1 row created.

SQL> INSERT INTO Orders VALUES (504, 104, NULL, SYSDATE, 799.00, 'Pending');
1 row created.

SQL> INSERT INTO Orders VALUES (505, 105, NULL, SYSDATE, 1599.00, 'Delivered');
1 row created.

SQL> COMMIT;
Commit complete.
```

7. ORDER DETAILS INSERTION

```
SQL> INSERT INTO Order_Details VALUES (601, 501, 101, 301, 2, 699.00);
1 row created.

SQL> INSERT INTO Order_Details VALUES (602, 502, 102, 302, 1, 749.00);
1 row created.

SQL> INSERT INTO Order_Details VALUES (603, 503, 103, 303, 2, 649.50);
1 row created.

SQL> INSERT INTO Order_Details VALUES (604, 504, 104, 304, 1, 799.00);
1 row created.

SQL> INSERT INTO Order_Details VALUES (605, 505, 105, 305, 2, 799.50);
1 row created.

SQL> COMMIT;
Commit complete.
```

8. DATA RETRIEVAL

8.1 DISPLAY ORDERS RECORDS

```
SQL> SELECT * FROM Orders;
```

ORDER_ID	CUSTOMER_ID	COUPON_ID	ORDER_DAT	TOTAL_AMOUNT

ORDER_STATUS				

501	101		26-SEP-26	1398
Pending				

502	102		26-SEP-26	749
Shipped				

503	103		26-SEP-26	1299
Delivered				

ORDER_ID	CUSTOMER_ID	COUPON_ID	ORDER_DAT	TOTAL_AMOUNT

ORDER_STATUS				

504	104		26-SEP-26	799
Pending				

505	105		26-SEP-26	1599
Delivered				

8.2 DISPLAY ORDER DETAILS RECORDS

```
SQL> SELECT * FROM Order_Details;
```

ORDER_DETAIL_ID	ORDER_ID	PRODUCT_ID	SELLER_ID	QUANTITY	UNIT_PRICE

601	501	101	301	2	699
602	502	102	302	1	749
603	503	103	303	2	649.5
604	504	104	304	1	799
605	505	105	305	2	799.5

9. ORDER MODIFICATION OPERATIONS

9.1 MODIFY ORDER AMOUNT

9.2 MODIFY ORDER DATE

```
SQL> UPDATE Orders SET Total_Amount = 1699.00 WHERE Order_ID = 501;
1 row updated.

SQL> UPDATE Orders SET Order_Date = TO_DATE('20-09-2026', 'DD-MM-YYYY') WHERE Order_ID = 501;
1 row updated.

SQL> COMMIT;
Commit complete.

SQL> SELECT * FROM Orders;

  ORDER_ID CUSTOMER_ID COUPON_ID ORDER_DAT TOTAL_AMOUNT
-----
ORDER_STATUS
-----
      501         101          20-SEP-26          1699
Pending
      502         102          26-SEP-26           749
Shipped
      503         103          26-SEP-26          1299
Delivered

  ORDER_ID CUSTOMER_ID COUPON_ID ORDER_DAT TOTAL_AMOUNT
-----
ORDER_STATUS
-----
      504         104          26-SEP-26           799
Pending
      505         105          26-SEP-26          1599
Delivered
```

10. ORDER DETAILS MODIFICATION

```
SQL> UPDATE Order_Details SET Unit_Price = 799.00 WHERE Order_Detail_ID = 601;
1 row updated.

SQL> UPDATE Order_Details SET Quantity = 3 WHERE Order_Detail_ID = 601;
1 row updated.

SQL> COMMIT;
Commit complete.

SQL> SELECT * FROM Order_Details;

ORDER_DETAIL_ID  ORDER_ID  PRODUCT_ID  SELLER_ID  QUANTITY  UNIT_PRICE
-----
          601         501         101         301           3           799
          602         502         102         302           1           749
          603         503         103         303           2          649.5
          604         504         104         304           1           799
          605         505         105         305           2          799.5
```

11. CUSTOMER ORDER HISTORY REPORT

```
SQL> SELECT
  2     O.Customer_ID,
  3     O.Order_ID,
  4     O.Order_Date,
  5     OD.Product_ID,
  6     OD.Seller_ID,
  7     OD.Quantity,
  8     OD.Unit_Price,
  9     O.Total_Amount,
10     O.Order_Status
11 FROM Orders O
12 JOIN Order_Details OD
13 ON O.Order_ID = OD.Order_ID
14 ORDER BY O.Customer_ID, O.Order_Date;
```

CUSTOMER_ID	ORDER_ID	ORDER_DAT	PRODUCT_ID	SELLER_ID	QUANTITY	UNIT_PRICE

TOTAL_AMOUNT ORDER_STATUS						

101	501	20-SEP-26	101	301	3	799
1699		Pending				
102	502	26-SEP-26	102	302	1	749
749		Shipped				
103	503	26-SEP-26	103	303	2	649.5
1299		Delivered				

CUSTOMER_ID	ORDER_ID	ORDER_DAT	PRODUCT_ID	SELLER_ID	QUANTITY	UNIT_PRICE

TOTAL_AMOUNT ORDER_STATUS						

104	504	26-SEP-26	104	304	1	799
799		Pending				
105	505	26-SEP-26	105	305	2	799.5
1599		Delivered				

12. CUSTOMER - WISE ORDER SUMMARY

```
SQL> SELECT
  2     C.Customer_ID,
  3     C.First_Name,
  4     C.Last_Name,
  5     COUNT(O.Order_ID) AS Total_Orders,
  6     SUM(O.Total_Amount) AS Total_Amount
  7 FROM Customers C
  8 JOIN Orders O
  9 ON C.Customer_ID = O.Customer_ID
 10 GROUP BY
 11     C.Customer_ID,
 12     C.First_Name,
 13     C.Last_Name
 14 ORDER BY C.Customer_ID;
```

CUSTOMER_ID	FIRST_NAME	LAST_NAME	TOTAL_ORDERS	TOTAL_AMOUNT
101	Nikitha	R	1	1699

102	Priya	Sharma	1	749
-----	-------	--------	---	-----

103	Arun	Kumar	1	1299
-----	------	-------	---	------

CUSTOMER_ID	FIRST_NAME	LAST_NAME	TOTAL_ORDERS	TOTAL_AMOUNT
104	Rahul	Verma	1	799

105	Sneha	Patel	1	1599
-----	-------	-------	---	------